



THE UNIVERSITY OF
MELBOURNE

Database Architecture and Administration

Database Systems & Information Modelling
INFO90002

Week 8 – DBA

Presented by Greg Wadley

Content by: Dr Tanya Linden, Dr Renata Borovica-Gajic,
David Eccles, Dr Simon D'Alfonso, Dr Greg Wadley





This lecture discusses

The 'Database Administrator' role

Architecture of a DBMS

- components of a DBMS
- how a transaction is processed

Performance improvement

- concepts
- indexes – what they do, when to use them.

The DBA role

Database Administrator

TTG Recruitment

Brisbane QLD (Hybrid)

A hybrid work model and the chance to work on complex, business-critical database platforms within a stable public sector environment

9d ago



Database Administrator

Osborne Richardson

Brisbane QLD (Hybrid)

A large state government department is looking for an experienced Oracle and SQL DBA to join their team for 12 months.

9d ago



The DBA role

Primarily concerned with “maintenance” / “ops” phase

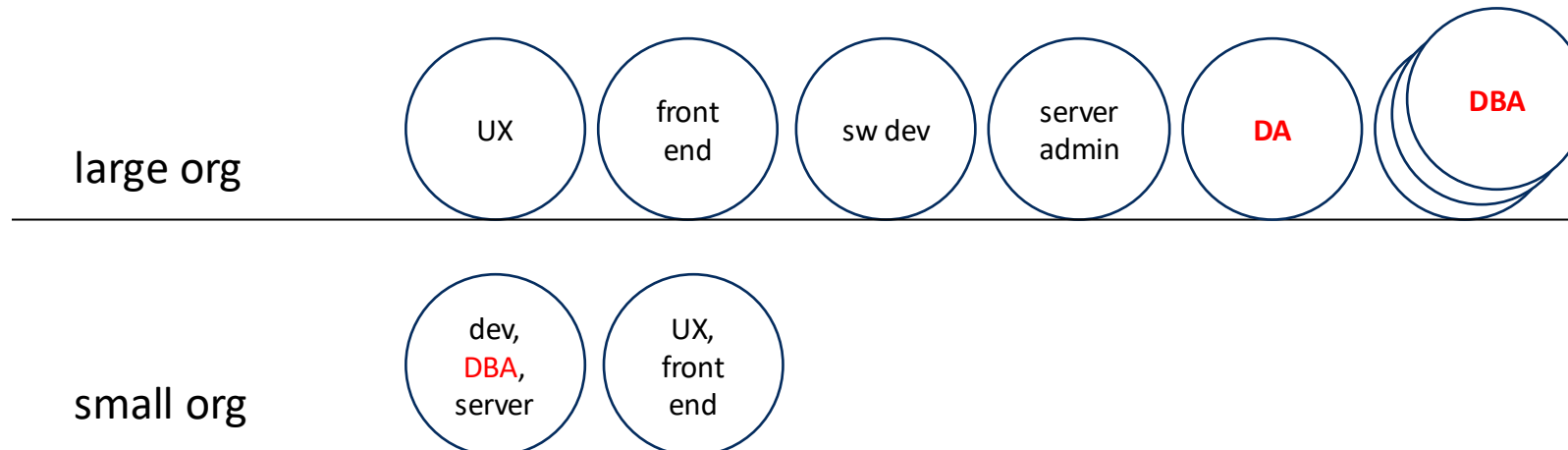
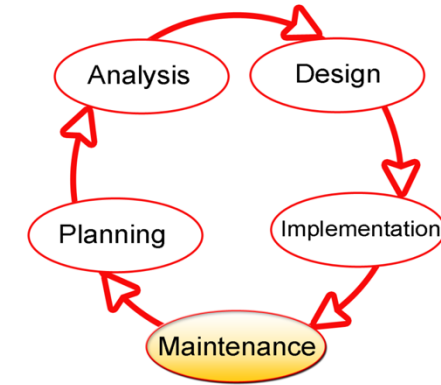
but should be consulted during all phases of development

“Database Administrator” or “DBA” - often framed as a “job” or a “person”

Large companies – many DBAs

Small company - a software developer might be the DBA

DBA role can be made redundant by Cloud-based DBMS or “data as a service”





Data and Database Administration

Data Administrator (aka CDO)

(management role)

- Data policies, procedures and standards
- Compliance with legislation (EU GDPR, Aus Privacy Act)
- Compliance with company policy (e.g. Uni of Melbourne privacy policy)
- Planning
- Data conflict resolution
- Managing information repository
- Internal marketing and education

Database Administrator

(technical role)

- analyze and design DB
- select DBMS / tools / vendor
- install and upgrade DBMS
- tune DBMS performance
- manage security, privacy, integrity
- backup and recovery

Data Administrator role

- Data conflict resolution
 - Conflict resolution may occur in e.g. customer management systems. For example, salespeople can maintain customer address information in different databases in a replication environment. Should a conflict arise, the system can resolve the conflicting updates by applying the most recent update to a record
- Managing information repository
 - A corporate repository is a place where a large amount of a **corporation's information** is stored, typically for the long term. Corporate repositories are **not made public**. Only users with specific roles and permissions may access documents
- Internal marketing and education
 - Internal marketing is the **promotion of a company's objectives, products and services** to employees within the organization. The purpose is to increase employee engagement with the company's goals and foster brand advocacy.

Summary: DA Versus DBA

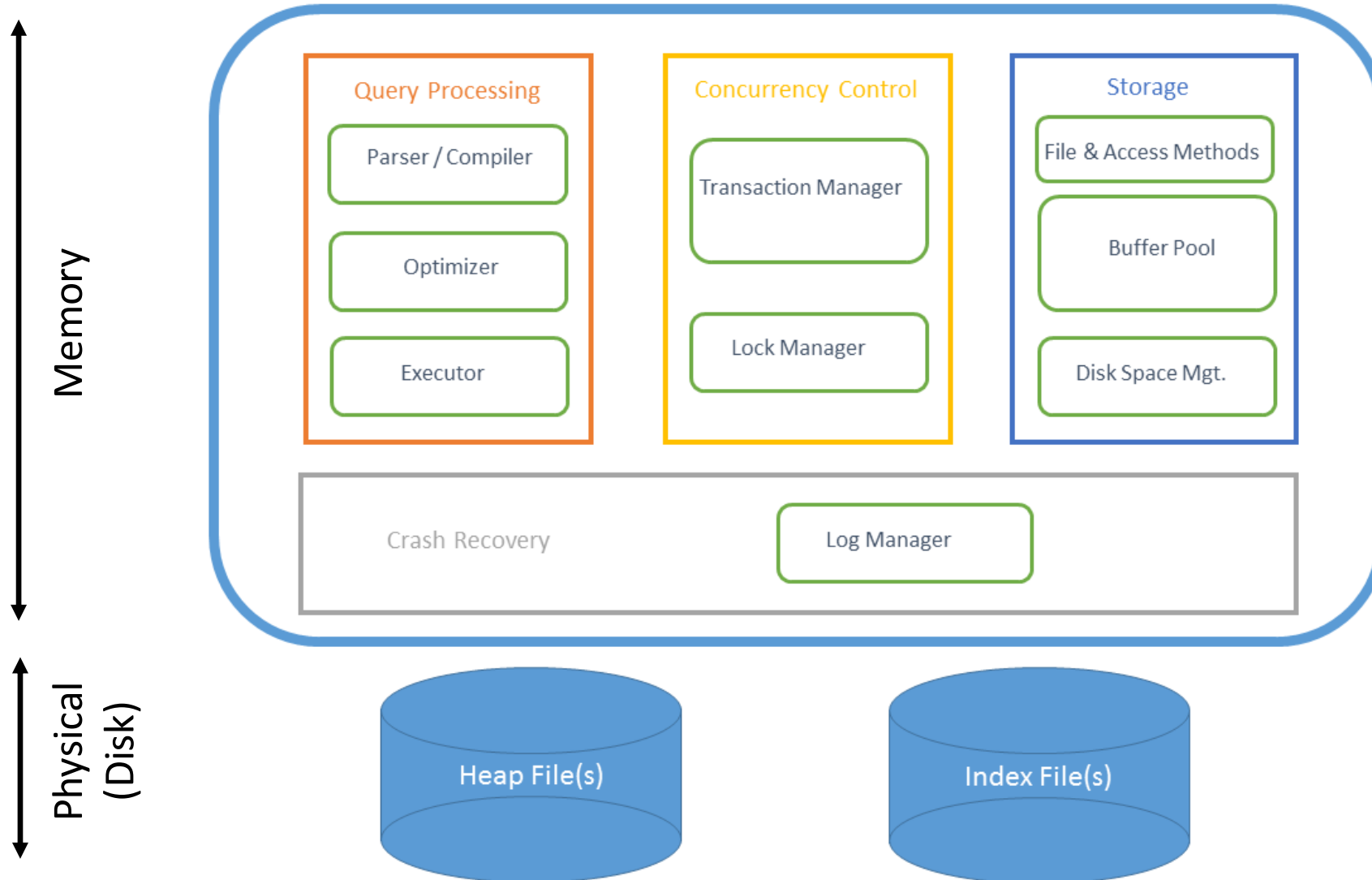
DATA ADMINISTRATOR (DA)	DATABASE ADMINISTRATOR (DBA)
Strategic planning	Control and supervision
Sets long-term goals	Executes plans to reach goals
Sets policies and standards	Enforces policies and procedures Enforces programming standards
Broad scope	Narrow scope
Long term	Short term (focus on daily operations)
Managerial orientation	Technical orientation
DBMS-independent	DBMS-specific



Database Management System

Architecture of a Database Management System (DBMS)

DBMS Architecture: overview



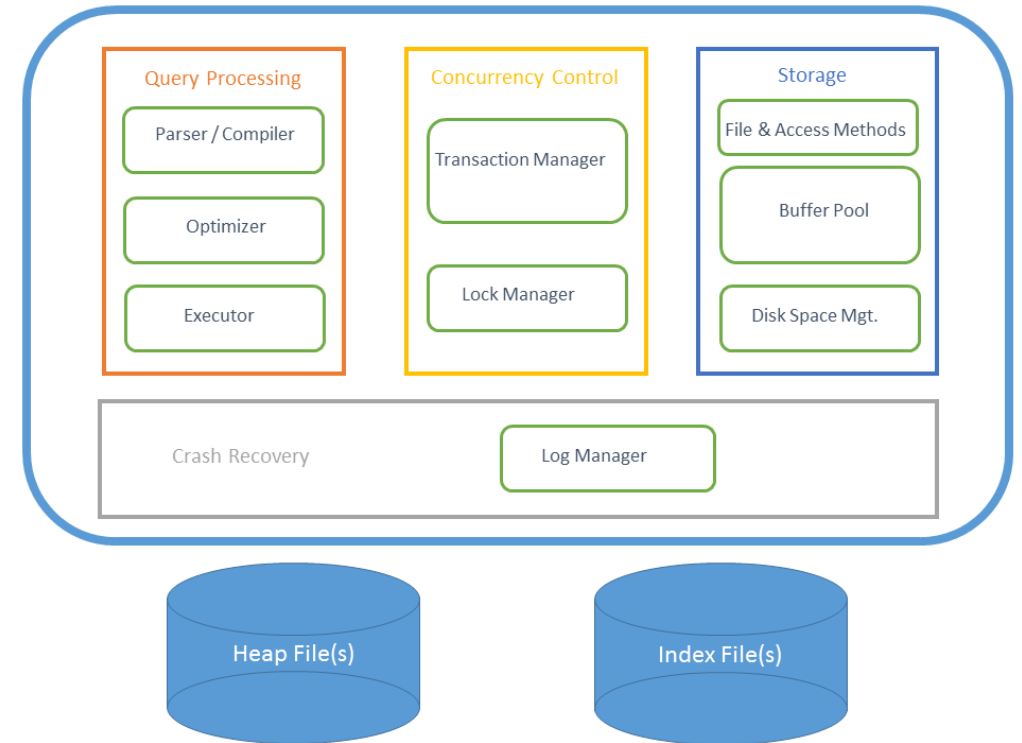
Server architecture

A Database Management System (DBMS) consists of:

- processes running in the CPU
- temporary storage in Main Memory (RAM)
- persistent storage on Hard Disk

The DBMS needs to provide features such as:

- data storage
- fast performance
- concurrent access by multiple users
- reliability, availability, recoverability



Caching Data in Memory

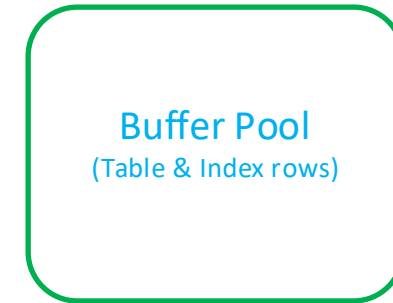
DBMS is complex (partly) because data is both stored on disk and cached in memory.

RAM and Disk have complementary characteristics:

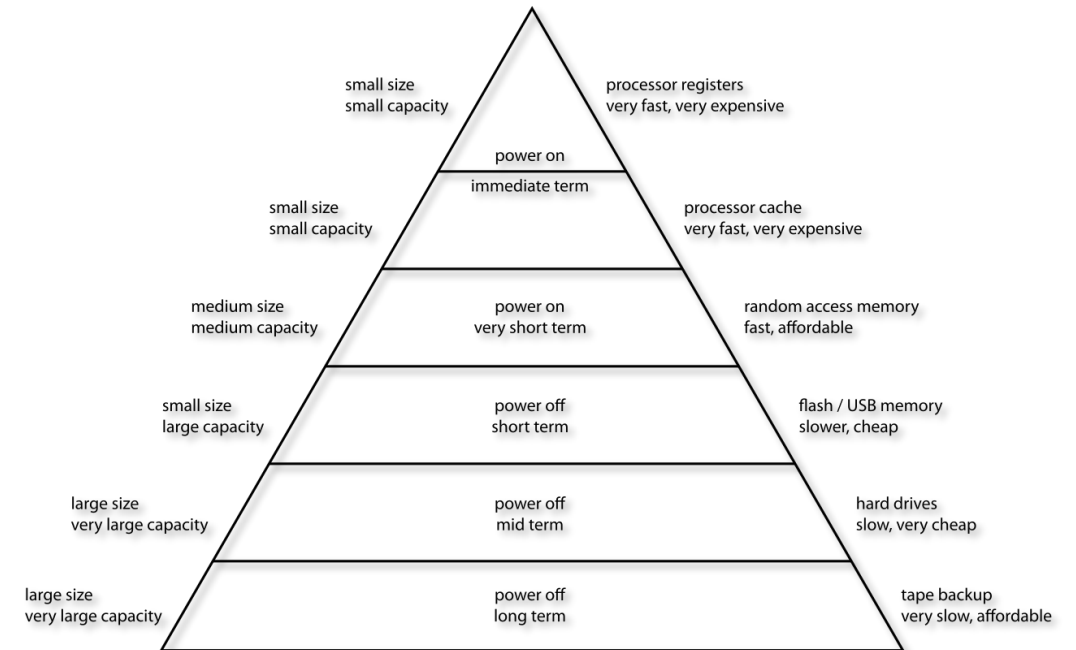
- RAM is fast, but contents are lost in a crash
- Disk is slow, but permanent

Therefore:

- Persist data on disk
- Cache data in memory
- Minimize reading from disk
- Write data to disk when it changes

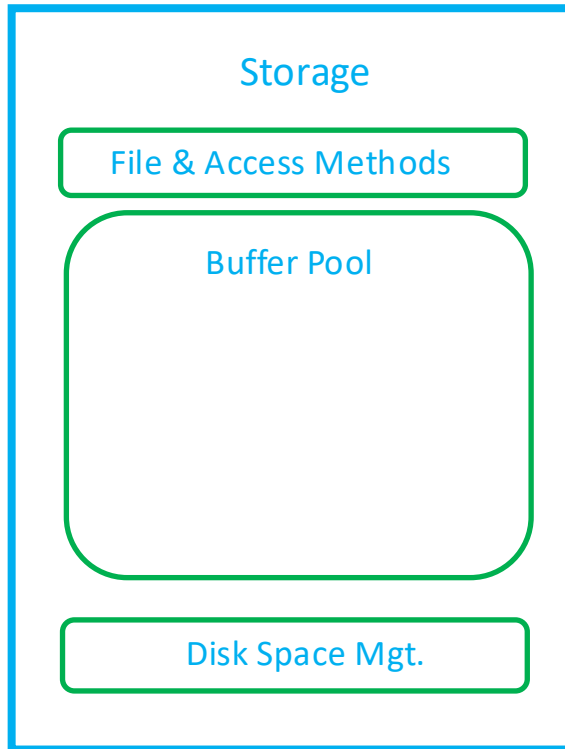


Computer Memory Hierarchy



https://en.wikipedia.org/wiki/Memory_hierarchy

Data Storage



File and Access Methods

- Data move from Disk to Memory and back to Disk

Buffer Pool

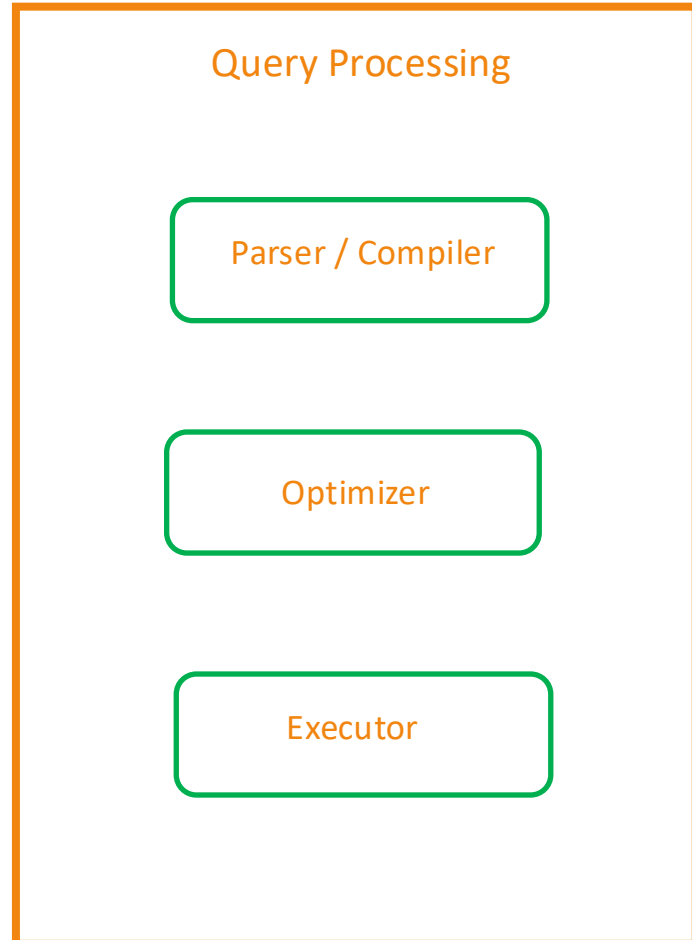
- Area of main memory used to cache data:
 - Row data
 - Index data

Buffer manager minimizes disk access by keeping needed frequently-used data in the buffer

Disk Space Management

- Allow tables (and rows) to grow

Query Processing



Parsing

- Check SQL syntax
- Check User Permissions

Optimising

- Create Execution Plan
- Evaluate indexes vs table scans
- Minimize cost (time)

Execution

- Ensure ACID transactions
- Atomic: All rows succeed, or all fail
- Move data between disk and cache as required

Query Processing

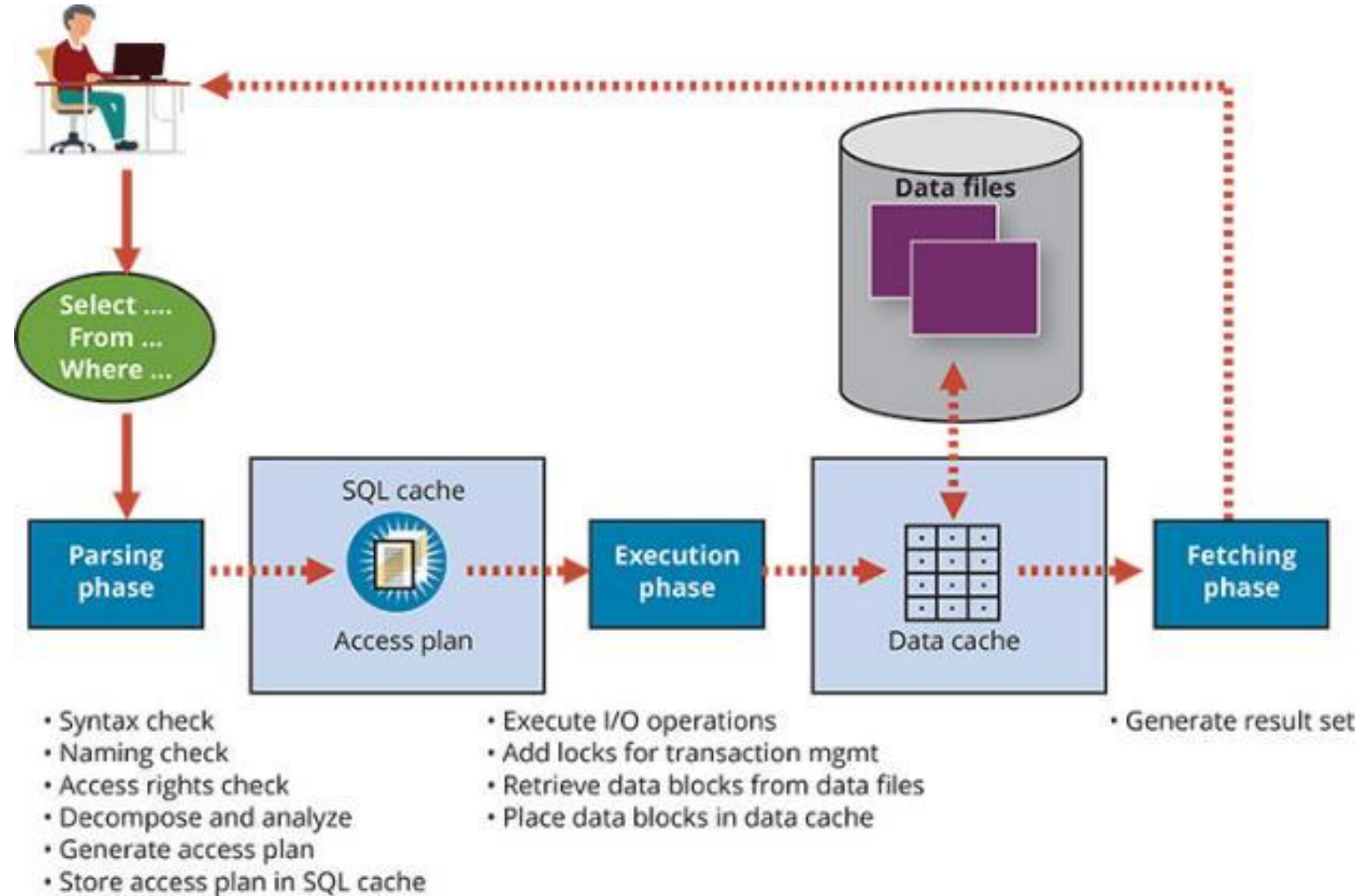
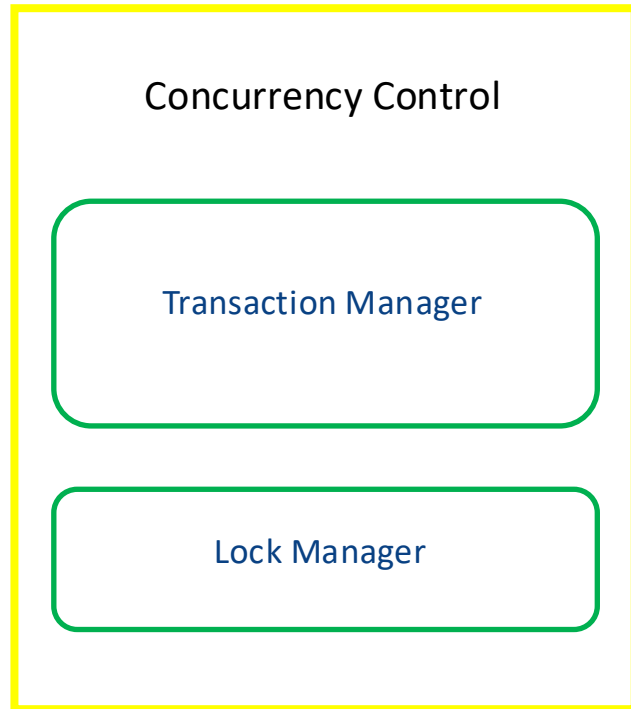


Figure 11.2 Query Processing (Coronel & Morris, 2023)

Concurrency Control



Manages access by multiple users.

Transaction Manager handles:

- which DBMS user / process wants
- which resource

Lock Manager maintains a list of what resources are locked by, and requested by, which users.

Log Manager

Log Manager records all changes in logs:

- UNDO / ROLLBACK information
- (old values, before they were changed)
- Supports rollback + isolation
- REDO information
- (record changes that were made to data, track which changes are not yet on disk)
- Includes Data Dictionary Changes

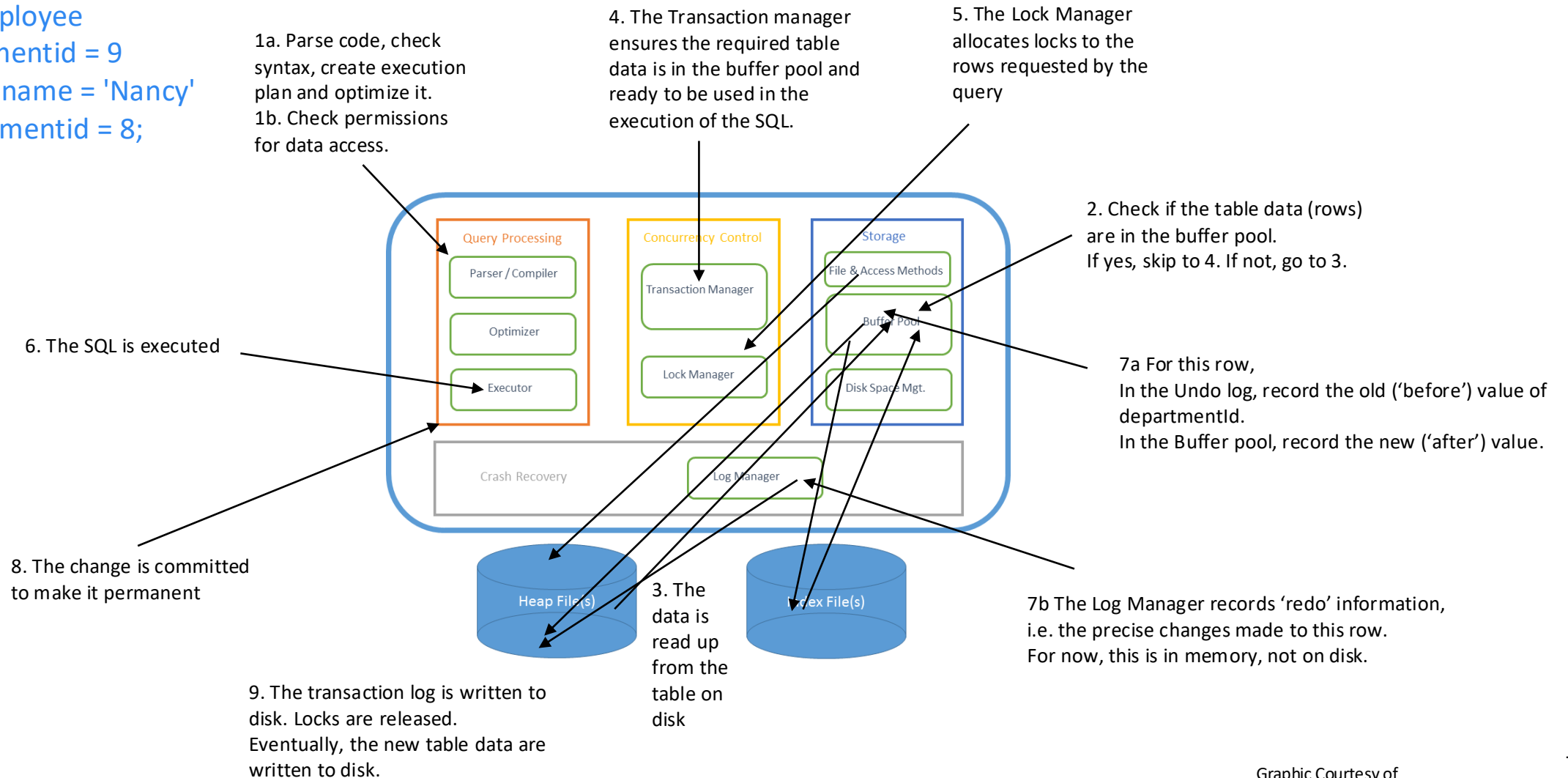
Crash Recovery

Log Manager

Changes are logged, *in addition to* the table data changing.

Processing a Transaction

UPDATE employee
SET departmentid = 9
WHERE firstname = 'Nancy'
AND departmentid = 8;



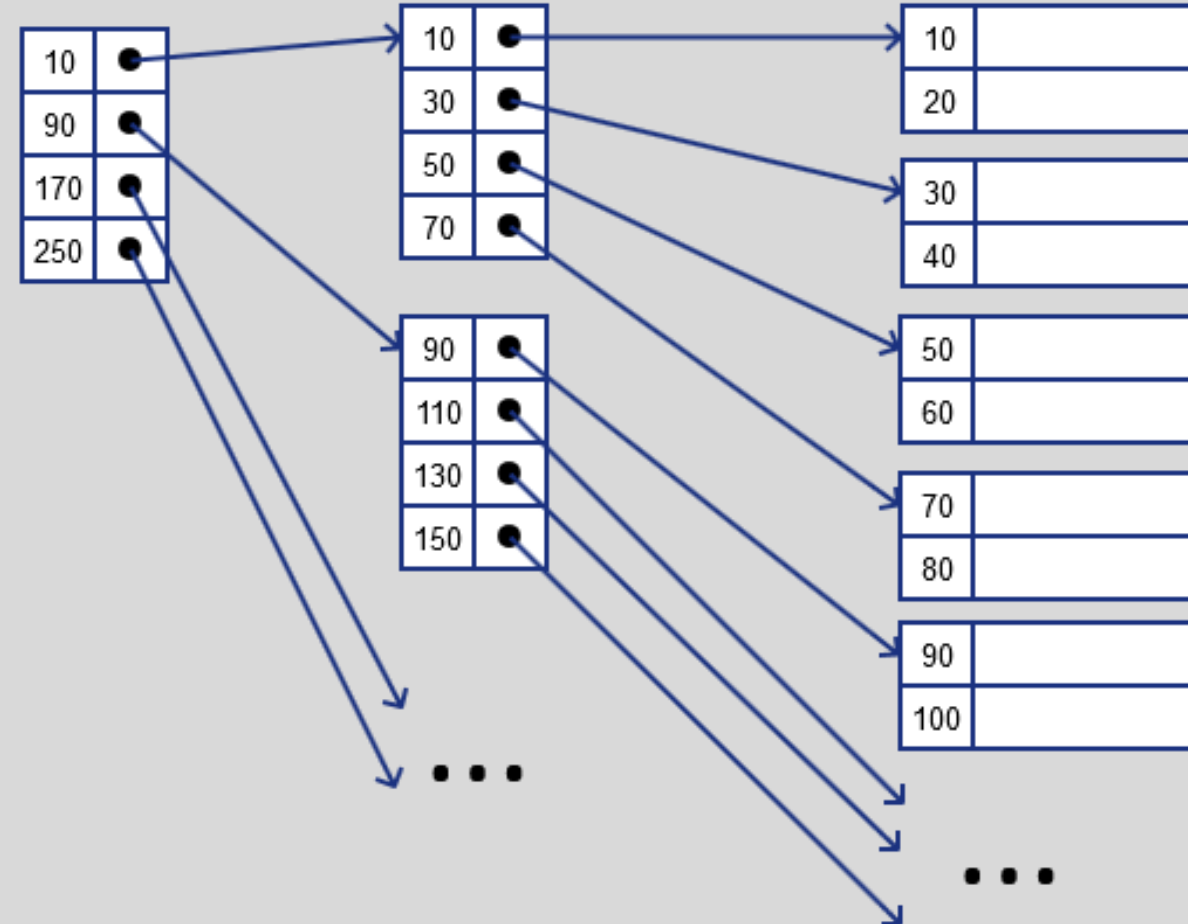
What affects database performance?

- Caching data in memory (buffer pool)
- Placement of data files across disc drives
- Fast reliable storage (e.g. SSD, RAID)
- Database replication and server clustering
- Use of indexes to speed up searches and joins
- Good choice of data types (especially PKs)
- Good program logic (no long running transactions)
- Code that avoids deadlocks
- Good query execution plans



THE UNIVERSITY OF
MELBOURNE

Indexing





Indexes

247	128	117	391	404	533	313	519	421	105	335	291	597	369	279
204	327	308	544	338	166	38	299	35	76	250	332	237	40	446
119	85	141	466	124	352	581	478	364	590	149	272	593	424	9
229	399	95	17	256	282	569	137	226	129	532	241	146	235	213
261	188	540	151	429	206	219	216	592	62	576	599	56	528	554
314	414	316	527	81	288	74	575	197	531	578	97	458	351	99
12	409	281	507	566	329	349	134	427	198	212	118	555	47	202
570	304	167	66	448	114	111	260	72	264	562	183	23	476	136
559	365	377	498	456	143	543	172	78	302	596	534	334	195	51
469	385	238	67	549	169	267	127	315	20	189	270	106	2	560
488	357	552	236	435	395	14	443	140	159	463	587	1	70	411
64	7	402	513	179	485	346	298	171	152	186	225	25	86	21
384	438	295	522	482	185	269	461	44	471	493	294	432	158	274
474	516	310	336	564	232	415	495	447	122	115	451	537	480	553
222	341	387	41	491	462	92	253	48	165	323	203	32	155	328
211	381	354	305	57	572	244	31	325	200	557	500	396	356	380
150	209	192	389	28	510	126	457	181	376	297	401	584	502	162
83	545	130	431	430	367	147	524	284	108	358	394	504	406	177
321	454	156	547	440	287	405	89	373	588	444	419	343	370	132
318	37	4	175	317	276	54	417	59	71	368	259	102	164	20 361

Imagine that this is a list of employee ids.

Do these employees exist?

- 502
- 91

It is hard and slow work to **sequentially** search through **every** item of data.



Indexes

1	44	85	127	159	197	237	282	318	357	395	432	474	524	560
2	47	86	128	162	198	238	284	321	358	396	435	476	527	562
4	48	89	129	164	200	241	287	323	361	399	438	478	528	564
7	51	92	130	165	202	244	288	325	364	401	440	480	531	566
9	54	95	132	166	203	247	291	327	365	402	443	482	532	569
12	56	97	134	167	204	250	294	328	367	404	444	485	533	570
14	57	99	136	169	206	253	295	329	368	405	446	488	534	572
17	59	102	137	171	209	256	297	332	369	406	447	491	537	575
20	62	105	140	172	211	259	298	334	370	409	448	493	540	576
21	64	106	141	175	212	260	299	335	373	411	451	495	543	578
23	66	108	143	177	213	261	302	336	376	414	454	498	544	581
25	67	111	146	179	216	264	304	338	377	415	456	500	545	584
28	70	114	147	181	219	267	305	341	380	417	457	502	547	587
31	71	115	149	183	222	269	308	343	381	419	458	504	549	588
32	72	117	150	185	225	270	310	346	384	421	461	507	552	590
35	74	118	151	186	226	272	313	349	385	424	462	510	553	592
37	76	119	152	188	229	274	314	351	387	427	463	513	554	593
38	78	122	155	189	232	276	315	352	389	429	466	516	555	596
40	81	124	156	192	235	279	316	354	391	430	469	519	557	597
41	83	126	158	195	236	281	317	356	394	431	471	522	559	599

Imagine that this is a list of employee ids.

Do these employees exist?

- 502
- 91

The search is **much easier** when the values are stored **in order**.

Sequential search is SLOW

Table scan: Read each record one by one from the beginning of the table until the required record is found.

- If there are 100,000 records
- On average, a search would require 50,000 disk accesses !

We could keep the table sorted in order ...
But that will get very expensive when we insert or update records.

The solution is to create an *Index*:

- Maps search field to record location
- Ordered by search field
- Small and easy to maintain

An Index makes searching (much) faster

Index is **much smaller** than the table.

Searching the index is **very fast** – for searches on that column.

Some work is required to **maintain** the index as the table changes.

Key	RowLoc
13	2
23	13
27	3
47	12
56	10
87	8
88	4
109	1
136	7
145	6
215	9
289	11
340	5

RowLoc	Empld	FirstName	Surname	Dept
1	109	Bronwyn	Harris	Mktg
2	13	Dave	Rigg	IT
3	112	Debra	Shaddock	Admin
4	88	Jenny	Black	Mktg
5	340	Karen	Charles	Admin
6	145	Lisa	Church	IT
7	136	Naomi	Allen	Admin
8	87	Nick	Taylor	Admin
9	215	Peter	Mills	Mktg
10	56	Steve	Avery	IT
11	289	Susan	Armstrong	Admin
12	47	Terry	Ymer	Admin
13	23	Tim	Wade	Admin

Searching the Index

Because the index is in order, the DBMS can use efficient search methods such as:

Binary Search

A Binary Search repeatedly **splits the index in half** until a match is (not) found.

Start at the middle position,

The value being searched for is either

- The current key value OR
- A key value higher in the list OR
- A key value lower in the list

Repeat until the required index record is found.

Key	RowLoc
13	2
23	13
27	3
47	12
56	10
87	8
88	4
109	1
136	7
145	6
215	9
289	11
340	5

Searching the Index

Once a match has been found, the RDBMS can **quickly retrieve** the appropriate record

Key	RowLoc	RowLoc	Empld	FirstName	Surname	Dept
13	2	1	109	Bronwyn	Harris	Mktg
23	13	2	13	Dave	Rigg	IT
27	3	3	112	Debra	Shaddock	Admin
47	12	4	88	Jenny	Black	Mktg
56	10	5	340	Karen	Charles	Admin
87	8	6	145	Lisa	Church	IT
88	4	7	136	Naomi	Allen	Admin
109	1	8	87	Nick	Taylor	Admin
136	7	9	215	Peter	Mills	Mktg
145	6	10	56	Steve	Avery	IT
215	9	11	289	Susan	Armstrong	Admin
289	11	12	47	Terry	Ymer	Admin
340	5	13	23	Tim	Wade	Admin

Inserting data into a simple Index

When a table has a **Primary Key**, an **Index** for the PK is **automatically** created.

As each row is **added** to the table, the DBMS automatically **inserts** an entry into the index: search **key** + **row location**.

Insert 1st row. Employee 109

- A **row is inserted** into the Employee table
- The **search key** and **row location** is added to the PK index

Table				
RowLoc	Empld	FirstName	Surname	Dept
1	109	Bronwyn	Harris	Mktg

Index	
Key	RowLoc
109	1

Inserting data into a simple Index

Insert a 2nd row for employee 13:

A row is **added to** the Employee table.

- The **search key** and **row location** is added to the PK index
- The PK Index is **automatically maintained** in PK sequence

Table				
RowLoc	Empld	FirstName	Surname	Dept
1	109	Bronwyn	Harris	Mktg
2	13	Dave	Rigg	IT

Index	
Key	RowLoc
13	2
109	1

Inserting data into a simple Index

Insert a 3rd row, for employee 27

A row is **added to** the Employee table.

- The **search key** and **row location** is added to the PK index
- The PK Index is **automatically maintained** in PK sequence

Table				
RowLoc	Empld	FirstName	Surname	Dept
1	109	Bronwyn	Harris	Mktg
2	13	Dave	Rigg	IT
3	27	Debra	Shaddock	Admin

Index	
Key	RowLoc
13	2
27	3
109	1

The cost of creating Indexes

Why not index **every** field?:

- **Disk Space** - a minor consideration. Disk space is cheap.
- **Update-Time** - a **major** consideration.
 - When records in a table are inserted, changed, deleted:
 - **Every index on that table must be updated**
so that each index remains in key sequence
 - **Each change takes time to complete**

Too many indexes can create **unacceptable** delays.

When to create indexes

For each table, consider placing an index on columns that are:

- *queried frequently* (used in WHERE clauses)
- used for *joins* (PK to FK)
- primary *keys* (automatic in most DBMS)
- foreign *keys* (automatic in MySQL)
- *unique* columns (automatic in most DBMS)

Index large tables only - small tables do not require indexes.

Index only if you frequently retrieve less than about 15% of the rows.

DBMS Optimiser – when will an index be used?

DBMS systems have **optimisers** that choose the most **efficient** way of accessing data.

Multiple relevant indexes may exist, but it is the **DBMS optimiser that decides** if it is of benefit to use the indexes.

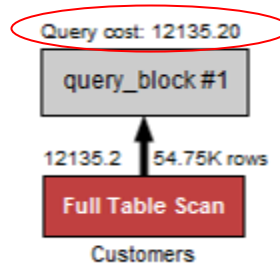
- Imagine we have a table with a **small number of rows**, and indexes based on the *EmployeeId* and *Surname* fields.
- Imagine that it only takes one or two disk reads to load the entire table into memory,
- **It may be quicker** to load all rows into memory and scan through them, compared to reading the Indexes and then retrieving the appropriate row(s) from disk.

Choosing the best Execution Plan

The best execution plan has the lowest “cost” (time to run the query)

```
SELECT cust_first_name, cust_last_name,
       cust_marital_status, cust_city, cust_income_level
FROM Customers
WHERE cust_last_name = 'Parkburg'
AND cust_first_name = 'Peter'
AND cust_city = 'Trafford';
```

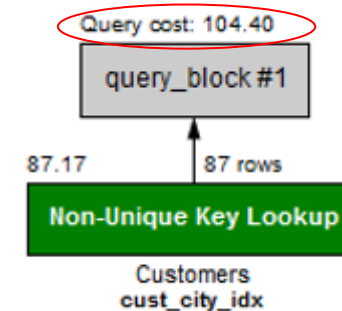
	cust_first_name	cust_last_name	cust_marital_status	cust_city	cust_income_level
	Peter	Parkburg	single	Trafford	H: 150.000 - 169.999



Index on *where* field improves cost:

```
CREATE INDEX cust_city_idx
ON Customers(cust_city);
```

```
SELECT cust_first_name, cust_last_name,
       cust_marital_status, cust_city, cust_income_level
FROM Customers
WHERE cust_last_name = 'Parkburg'
AND cust_first_name = 'Peter'
AND cust_city = 'Trafford';
```





What's examinable

- What a DBA and Data Administrator do
 - and the differences between the roles
- Database Architecture
 - components & their roles
- What affects database performance
 - Caching; Indexes; Data types; Query Execution plans; Efficient code;
- When to create an index



THE UNIVERSITY OF
MELBOURNE

Database Architecture and Administration

Database Systems & Information Modelling
INFO90002

Week 8 – DBA

Presented by Greg Wadley

Content by: Dr Tanya Linden, Dr Renata Borovica-Gajic,
David Eccles, Dr Simon D'Alfonso, Dr Greg Wadley

